Unit 4 – Analytical Questions

Network Security: IPSec and Email Security

Question 1: *Analyze the IPSec architecture and implement a secure communication model that demonstrates how AH and ESP protect IP packets against various network attacks.*

1.1 IPSec Architecture Overview

Internet Protocol Security (IPSec) is a suite of protocols operating at the network layer (Layer 3) that provides confidentiality, integrity, authentication, and anti-replay protection for IP traffic. It is not a single protocol but a framework composed of several cooperating components:

- Authentication Header (AH) – provides connectionless integrity, data-origin authentication, and anti-replay protection for the IP packet.
- Encapsulating Security Payload (ESP) – provides confidentiality (encryption) in addition to the integrity and authentication services offered by AH.
- Security Association (SA) – a unidirectional logical relationship that defines the security parameters (algorithm, key, mode) between two communicating entities.
- Internet Key Exchange (IKE) – automates the negotiation of SAs and cryptographic keys.
- Security Policy Database (SPD) and Security Association Database (SAD) – control which traffic is protected and how.

1.2 Secure Communication Model (Design)

The model below simulates two hosts, Host A and Host B, communicating over an untrusted network. Traffic between them is intercepted by an attacker who attempts (a) eavesdropping, (b) packet tampering, and (c) replay of captured packets.

Stage	Action	Protection Applied
1. Policy lookup	Host A checks SPD for the destination	Determines AH, ESP, or both are required
2. SA establishment	IKE negotiates keys and algorithms	Produces a bidirectional pair of SAs
3. Outbound processing	Packet is authenticated (AH) and/or encrypted (ESP)	ICV computed; payload encrypted if ESP used
4. Transmission	Packet sent across untrusted network	Attacker sees ciphertext (ESP) or a protected but visible payload (AH only)
5. Inbound processing	Host B looks up SA using SPI, verifies ICV, decrypts if needed	Tampered or replayed packets are dropped

1.3 How AH and ESP Defeat Specific Attacks

Attack	Mechanism Exploited	IPSec Countermeasure
Eavesdropping / sniffing	Plaintext payload visible on the wire	ESP encrypts the payload using a symmetric cipher (e.g., AESCBC/GCM), so intercepted packets are unreadable
Packet tampering / man-in-themiddle modification	Attacker alters header or payload in transit	AH and ESP both compute an Integrity Check Value (ICV) over the packet using HMAC (e.g., HMAC-SHA256); any modification causes ICV verification to fail at the receiver
IP spoofing	Attacker forges the source address	Data-origin authentication in AH/ESP proves the packet originated from a host holding the shared secret key, not just any host claiming that IP
Replay attack	Captured packet is retransmitted later	Sequence Number field, checked against a sliding anti-replay window at the receiver, rejects duplicate or out-of-window packets
Traffic analysis (partial)	Attacker infers communication patterns from headers	Tunnel-mode ESP hides the original inner IP header, reducing exposed metadata

1.4 Conceptual Implementation Notes

A working prototype (e.g., in Python using scapy, or via a Linux strongSwan/Libreswan setup) would proceed as follows:

1. Configure two virtual hosts with IPSec-capable stacks and define an SPD rule requiring ESP in tunnel mode for traffic between them.
2. Use IKEv2 (or a static pre-shared key for a simplified demo) to establish SAs with AES-256-GCM for encryption and HMAC-SHA-256 for integrity.
3. Send test traffic and capture it with Wireshark; verify the payload is unreadable (ESP) and that the ICV/SPI/sequence-number fields are present.
4. Simulate an attacker modifying one byte of a captured packet and replaying it; observe the receiver drop it due to ICV mismatch or replay-window rejection.

This demonstrates empirically that AH guarantees authenticity/integrity while ESP additionally guarantees confidentiality, and that both defeat the classic active/passive attacks listed above.

Question 2: *Compare and analyze the performance of AH and ESP in terms of confidentiality, integrity, authentication, packet overhead, and computational cost using experimental results.*

2.1 Feature Comparison

Property	Authentication Header (AH)	Encapsulating Security Payload (ESP)
Confidentiality	Not provided	Provided (payload encryption)
Property	**Authentication Header (AH)**	**Encapsulating Security Payload (ESP)**
Integrity	Provided (entire packet, incl. immutable IP header fields)	Provided (ESP header, payload, trailer — outer IP header not covered in transport mode)
Authentication	Provided (data-origin)	Provided (data-origin)
Anti-replay	Provided (sequence number)	Provided (sequence number)
NAT compatibility	Poor — AH integrity check breaks when NAT rewrites IP addresses	Good, especially with NAT-Traversal (UDP encapsulation)
IP protocol number	51	50

2.2 Packet Overhead

AH adds a fixed header of 24 bytes (with a 96-bit ICV) and contributes no ciphertext padding. ESP adds an 8-byte header, an ICV (commonly 12–16 bytes for GCM/HMAC-truncated output), a trailer with padding to align to the block size, and — because it encrypts — may add 0–15 bytes of padding for block ciphers such as AES-CBC. In tunnel mode both protocols add a complete new IP header (20 bytes for IPv4), roughly doubling header overhead compared to transport mode.

Mode/Protocol	Approx. Added Overhead (IPv4)	Notes
AH – Transport	~24 bytes	No encryption padding
ESP – Transport (AES-CBC)	~26–36 bytes	Includes IV, padding, trailer, ICV
ESP – Transport (AES-GCM)	~22–30 bytes	No separate ICV field; combined AEAD tag
AH – Tunnel	~44 bytes	Adds new outer IP header
ESP – Tunnel (AES-CBC)	~50–60 bytes	New IP header + ESP overhead

2.3 Computational Cost

AH is computationally lighter because it only performs one HMAC pass over the packet. ESP with encryption performs both a symmetric-cipher operation (AES) and a MAC/AEAD operation, so it is more CPU-intensive per packet. In practice, hardware AES-NI acceleration narrows this gap considerably; AEAD modes such as AES-GCM combine encryption and authentication in a single pass and are typically faster than the older HMAC + AES-CBC combination used by ESP historically.

Representative experimental results (typical of published IPSec benchmarking studies, illustrative figures):

Configuration	Throughput (approx., 1 Gbps link)	Latency added per packet
No IPSec (baseline)	~940 Mbps	~0 ms
AH (HMAC-SHA256)	~880–900 Mbps	0.02–0.05 ms
ESP (AES-CBC + HMAC-SHA1)	~750–800 Mbps	0.08–0.15 ms
ESP (AES-GCM, hardware-accelerated)	~870–900 Mbps	0.03–0.07 ms

2.4 Analysis

The results show a clear trade-off: AH offers lower overhead and computational cost but no confidentiality, making it unsuitable where data secrecy matters and largely obsolete for NATed networks. ESP, particularly with AEAD ciphers, delivers confidentiality plus integrity at a moderate performance cost that modern hardware absorbs well. For this reason, most production IPSec deployments today use ESP exclusively, with AH rarely deployed in isolation.

Question 3: *Design and evaluate a Security Association (SA) management system that supports multiple secure communication sessions and analyze its effectiveness in preventing replay and impersonation attacks.*

3.1 Design Goals

- Support many concurrent SAs across multiple peers and traffic selectors.
- Provide fast, unambiguous SA lookup on the inbound path.
- Support key lifetime limits and automatic rekeying.
- Detect and reject replayed or impersonated packets.

3.2 System Architecture

The SA management system consists of three logical databases and a control-plane daemon:

- Security Policy Database (SPD): ordered rule set mapping traffic selectors (source/destination address, protocol, port) to a required action — bypass, discard, or protect (with a reference to an SA bundle).
- Security Association Database (SAD): one entry per active SA, indexed by the triple {Security Parameter Index (SPI), destination address, protocol (AH/ESP)}. Each entry stores the encryption/authentication algorithms, keys, sequence-number counter, anti-replay window, and lifetime counters (time- and byte-based).
- IKE Daemon: negotiates new SAs, performs rekeying before lifetime expiry, and deletes/purges expired SAs from the SAD.

3.3 Supporting Multiple Sessions

Because SAs are unidirectional and identified by SPI + destination + protocol, the system can maintain an arbitrary number of simultaneous sessions simply by allocating a unique SPI per direction per peer per traffic class. A hash table keyed on this triple gives O(1) average-case lookup even with thousands of concurrent SAs, which is essential for a gateway serving many VPN tunnels.

3.4 Replay Protection Mechanism

Each SA maintains a monotonically increasing 32- or 64-bit sequence number on the sender side and a sliding replay window (default size 64) on the receiver side. When a packet arrives:

1. If the sequence number is higher than the highest seen, the packet is accepted and the window advances.
2. If it falls within the current window but has already been marked as received, it is rejected.
3. If it falls below the window's lower edge, it is rejected as a stale replay.

This design bounds an attacker to a very narrow, already-expired replay opportunity, and combined with the ICV check (which cryptographically binds the sequence number to the packet), sequence numbers cannot be forged independently of the payload.

3.5 Impersonation Protection

Impersonation is prevented at two levels: (1) SA establishment via IKE uses mutual authentication (pre-shared keys, digital certificates, or EAP), so only a peer holding the correct credential can complete negotiation and obtain a valid SPI/key pair; (2) per-packet data-origin authentication via HMAC/AEAD ensures that even if an attacker learns a valid SPI, they cannot produce a packet that verifies correctly without possessing the negotiated secret key.

3.6 Evaluation

Threat	Without SA Management	With Proposed SA Management
Replay of old packets	Accepted (no ordering enforcement)	Rejected by sliding-window check
Threat	**Without SA Management**	**With Proposed SA Management**
Impersonation without key	Possible if source IP is trusted	Rejected — ICV/AEAD verification fails
SA confusion across sessions	Risk of key/session mix-up	Eliminated via unique SPI-based indexing
Stale/compromised keys	Persist indefinitely	Bounded by lifetime-based automatic rekeying

Overall, the SA management system scales to many concurrent sessions with constant-time lookup and closes both the replay and impersonation attack surfaces by combining cryptographic authentication with stateful sequence tracking.

Question 4: *Implement and analyze IPSec Transport Mode and Tunnel Mode for a given network topology. Compare their security features, packet formats, and suitability for different networking scenarios.*

4.1 Reference Topology

Consider two scenarios: (a) Host-to-Host — two end systems, Host A and Host B, communicating directly across the Internet; (b) Gateway-to-Gateway — two private LANs, LAN 1 (behind Gateway G1) and LAN 2 (behind Gateway G2), connected via a site-to-site VPN across the public Internet.

4.2 Packet Format Comparison

Mode	Packet Structure (ESP example)	What Is Protected
Transport Mode	[Original IP Header][ESP Header][TCP/UDP + Data][ESP Trailer][ICV]	Only the payload (transport-layer segment); original IP header remains in the clear (except being covered by AH's integrity check when AH is used)
Tunnel Mode	[New IP Header][ESP Header][Original IP Header][TCP/UDP + Data][ESP Trailer][ICV]	The entire original IP packet, including its header, is encapsulated and encrypted/authenticated

4.3 Security Feature Comparison

Feature	Transport Mode	Tunnel Mode
Original source/destination visibility	Visible to intermediate routers	Hidden — only the new outer gateway addresses are visible
End-point requirement	Both communicating hosts must be IPSec-aware	Only the gateways need to implement IPSec; end hosts are unaware

Overhead	Lower (no extra IP header)	Higher (extra ~20 bytes IPv4 header)
Typical deployment	Host-to-host secure sessions (e.g., securing a specific application flow)	Site-to-site VPNs, remote-access VPNs
Traffic analysis resistance	Weaker — real endpoints visible	Stronger — internal topology hidden

4.4 Implementation Approach

For the host-to-host case, Transport Mode is configured directly on Host A and Host B (e.g., via IPSec policies in strongSwan/Windows IPSec) protecting a specific application's traffic. For the gateway-to-gateway case, Tunnel Mode is configured on G1 and G2; internal hosts on LAN 1/LAN 2 send ordinary unprotected packets to their gateway, which encapsulates them before sending across the Internet, and the remote gateway decapsulates and forwards them onto the destination LAN.

4.5 Suitability Analysis

- Transport mode suits scenarios needing lower overhead and where both endpoints already support IPSec — e.g., securing management traffic between two specific servers.
- Tunnel mode suits scenarios connecting whole networks or supporting remote-access clients, since it requires no changes to individual internal hosts and additionally conceals internal addressing from the public network.

In practice, nearly all commercial VPN products use Tunnel Mode because it is topology-flexible and easier to deploy at scale, while Transport Mode is reserved for narrower host-level use cases.

Question 5: *Develop a simplified Internet Key Exchange (IKE) mechanism and analyze how dynamic key management improves communication security compared to static key distribution.*

5.1 Simplified IKE Design (Two-Phase Model)

The simplified mechanism mirrors real IKEv2 but omits optional extensions, for clarity of analysis.

Phase 1 – Establish a secure, authenticated channel (IKE_SA):

1. Host A and Host B exchange Diffie–Hellman public values (g^a mod p, g^b mod p) together with nonces, over an initial unauthenticated exchange.
2. Both sides compute the shared Diffie–Hellman secret g^ab mod p and derive a set of symmetric keys (SK_e for encryption, SK_a for authentication) using a key-derivation function (e.g., HMAC-based KDF, as in IKEv2's PRF).
3. Each party authenticates the exchange (using a pre-shared key or digital signature over the exchange transcript) to prevent man-in-the-middle attacks, and confirms peer identity.

Phase 2 – Negotiate IPSec SAs (CHILD_SA):

1. Using the secure channel from Phase 1, the peers negotiate the actual IPSec SA parameters: encryption algorithm, integrity algorithm, SPI values, and traffic selectors.
2. A fresh session key for the IPSec SA is derived from the Phase-1 keying material plus new nonces, ensuring key separation between the control channel and the data channel.
3. Before the negotiated SA's lifetime expires, a rekey exchange (optionally with a new Diffie–Hellman exchange for Perfect Forward Secrecy) produces a new key, and the old SA is retired.

5.2 Dynamic vs. Static Key Distribution

Aspect	Static Key Distribution	Dynamic Key Management (IKE)
Key setup	Manually configured, identical key used indefinitely	Automatically negotiated per session via Diffie–Hellman
Exposure window	A single compromised key exposes all past and future traffic	Regular rekeying limits exposure to the current session only
Forward secrecy	None — one leaked key compromises all prior sessions	Achievable — ephemeral DH exchange per rekey means past sessions stay secure even if a later key leaks
Aspect	**Static Key Distribution**	**Dynamic Key Management (IKE)**
Scalability	Poor — every peer pair needs manual key provisioning ($O(n^2)$ key pairs)	Good — keys are derived on demand, no manual per-pair provisioning
Resistance to replay of the keyexchange process	N/A (no exchange to replay)	Nonces and sequence checks prevent replay of the negotiation itself
Operational burden	High — manual redistribution needed after suspected compromise	Low — automatic renegotiation and revocation

5.3 Analysis

Dynamic key management materially improves security along three dimensions. First, it removes the single-point-offailure inherent in a long-lived static key, since compromise of one session's derived key does not expose other sessions when Perfect Forward Secrecy is used. Second, it automates authentication of the exchange itself, closing the man-inthe-middle gap that plain, unauthenticated Diffie–Hellman would leave open. Third, it scales to large networks because keys are generated on demand rather than requiring administrators to pre-distribute and manage a key for every possible pair of communicating hosts. The cost is added negotiation latency and computational overhead for the DH exchange, which is a reasonable trade-off given the substantial security gains.

Question 6: *Analyze the structure of an email header to identify the actual sender, intermediate mail servers, authentication fields, and indicators of email spoofing or phishing attacks.*

6.1 Anatomy of an Email Header

A raw email header consists of an ordered set of fields added by each system the message passes through. Key fields include:

Field	Purpose	Spoofing Relevance
From	Display name and address the client shows to the user	Trivially forgeable — attacker can put any address here
Return-Path / Envelope-From	Address used for bounce messages, set by the sending MTA	Often differs from 'From' in spoofed mail; mismatch is a red flag
Received (multiple, in reverse chronological order)	Each mail server logs the hop it processed, including timestamps and originating IP	Reading bottom-to-top reveals the true originating server; forged headers often show inconsistent hostnames/IPs or missing hops

Message-ID	Unique identifier assigned by the originating system	Domain in the Message-ID that doesn't match the claimed sender's domain is suspicious
SPF (Sender Policy Framework) result	Verifies the sending server's IP is authorized for the claimed domain	'Fail' or 'softfail' strongly indicates spoofing
DKIM-Signature / AuthenticationResults	Cryptographic signature proving the message (or key parts) was not altered and originated from a domain holding the private key	A missing or invalid DKIM signature is a strong phishing indicator
DMARC result	Combines SPF/DKIM with domain alignment policy	'Fail' indicates the message does not comply with the domain owner's stated policy
Reply-To	Address replies are routed to	Often set to an attacker-controlled address different from 'From' in phishing emails

6.2 Tracing the True Sender

Because each relaying mail server prepends its own Received header, the header block should be read from bottom to top to reconstruct the actual path: the bottom-most Received header is closest to the true originating mail client/server, while the top-most is the last hop before delivery to the recipient's mailbox. Inconsistent hostnames (e.g., a Received line claiming to be from 'mail.bank.com' but showing an IP that resolves to an unrelated hosting provider) reveals the real origin regardless of what the From field claims.

6.3 Phishing/Spoofing Indicators — Summary Checklist

- From domain does not match the Return-Path or Message-ID domain.
- SPF = fail/softfail or DKIM = fail/none in the Authentication-Results header.
- DMARC alignment failure despite a 'protected' sending domain.
- Received chain shows an originating IP geographically or organizationally inconsistent with the claimed sender.
- Reply-To address differs from the displayed From address.
- Missing or malformed Message-ID, or reused Message-IDs across unrelated emails (bulk phishing indicator).

6.4 Analytical Conclusion

Header analysis is most reliable when it cross-validates multiple independent fields rather than trusting any single one, since the From field alone is fully attacker-controlled. Automated authentication results (SPF, DKIM, DMARC) provide the strongest cryptographic and policy-based evidence, while manual inspection of the Received chain is valuable for forensic tracing when those mechanisms are absent or inconclusive.

Question 7: ***Design and implement a Pretty Good Privacy (PGP)-based secure email system. Analyze the confidentiality, authentication, integrity, and non-repudiation services provided by your implementation.***

7.1 System Design

The system uses a hybrid cryptosystem, combining asymmetric key exchange with symmetric bulk encryption, following the classical PGP model:

Sending side (Sender S, Recipient R):

1. S computes a hash (SHA-256) of the message and signs the hash with S's RSA/DSA private key, producing a digital signature — this provides authentication and non-repudiation.
2. S generates a random one-time session key and encrypts the message plus signature with a symmetric cipher (AES-128/256) using this session key — this provides confidentiality.
3. S encrypts the session key itself with R's RSA public key and attaches it to the message.
4. The result is optionally Base64-encoded (radix-64) for safe transmission over text-based email systems, and compressed (ZIP) before encryption to improve efficiency and slightly reduce cryptanalytic patterns.

Receiving side:

1. R decrypts the session key using R's RSA private key.
2. R uses the recovered session key to decrypt the message and signature.
3. R computes a fresh hash of the decrypted message and verifies it against the signature using S's public key — confirming both integrity and authenticity.

7.2 Key Management

PGP uses a decentralized Web of Trust rather than a central certificate authority: each user maintains a public keyring of correspondents' public keys and a private keyring of their own key pairs, and trust in a given public key is established by other users' signatures (introductions) rather than a hierarchical CA.

7.3 Security Services Analysis

Service	Mechanism	How It Is Achieved
Confidentiality	Session-key symmetric encryption + RSA-wrapped session key	Only the holder of R's private key can recover the session key and thus the plaintext
Authentication	Digital signature over message hash using S's private key	Only S could have produced a signature verifiable with S's public key
Integrity	SHA-256 hash bound inside the signature	Any alteration of the message changes the hash, causing signature verification to fail
Non-repudiation	Signature is tied to S's uniquely held private key	S cannot credibly deny having sent a validly-signed message, since only S could have generated that signature

7.4 Evaluation

The design satisfies all four required services simultaneously by layering signing before encryption (sign-thenencrypt), which additionally hides the identity bound to the signature from any party lacking the session key. Its main practical limitation is key-distribution trust: without a central CA, the strength of the authentication service depends on the diligence of the Web of Trust, so a user who accepts an unverified public key undermines the authentication and non-repudiation guarantees regardless of the cryptography's correctness.

Question 8: *Implement and compare PGP and S/MIME for secure email communication. Analyze their differences with respect to certificate management, key distribution, scalability, and enterprise deployment.*

8.1 Overview

Both PGP and S/MIME provide confidentiality, integrity, authentication, and non-repudiation for email using hybrid (asymmetric + symmetric) cryptography, but they differ fundamentally in trust architecture and standardization.

8.2 Comparative Analysis

Dimension	PGP	S/MIME
Trust model	Decentralized Web of Trust — users vouch for each other's keys	Centralized hierarchical PKI — X.509 certificates issued by trusted Certificate Authorities (CAs)
Certificate management	No formal certificate; a public key with attached signatures ('introducers')	Formal X.509 certificates with defined validity periods, revocation (CRL/OCSP), and CA-issued chains of trust
Key distribution	Via public key servers, direct exchange, or key-signing parties	Via CA-issued certificates, often integrated with organizational
Dimension	**PGP**	**S/MIME**
		directory services (LDAP/Active Directory)
Scalability	Scales well informally among individuals/small communities but Web-of-Trust verification becomes unwieldy at large scale	Scales well in large organizations due to centralized CA issuance and automated certificate lifecycle management
Revocation	Manual key revocation certificates, propagation depends on keyserver sync	Standardized CRL/OCSP revocation checking, generally faster and more reliable
Enterprise deployment	Less common in enterprises; favored by individuals, journalists, opensource communities	Widely adopted in enterprises due to integration with existing PKI, Outlook/Exchange support, and compliance requirements
Interoperability	Requires both parties to use compatible PGP-capable clients/plugins	Broad native support in mainstream email clients (Outlook, Apple Mail, Thunderbird)

8.3 Implementation Notes

A PGP implementation (e.g., using GnuPG) requires each user to generate a key pair, publish the public key, and manually establish trust paths. An S/MIME implementation instead requires each user to obtain a certificate from an internal or public CA, which the enterprise can automate through certificate auto-enrollment, after which mainstream mail clients transparently sign and encrypt messages using the certificate.

8.4 Analysis and Recommendation

For individual users and communities without centralized IT infrastructure, PGP's Web of Trust avoids dependence on any single authority and works well at modest scale. For enterprises, S/MIME is generally preferable because it integrates with existing PKI, supports standardized revocation, and is natively supported by common mail clients — reducing user friction and administrative overhead. The trade-off is that S/MIME's security guarantee is only as strong as the trustworthiness of the issuing CA, whereas PGP concentrates trust decisions with individual users, which increases flexibility but also user responsibility.

Question 9: *Develop a machine learning-based email spam detection system using a suitable classification algorithm (e.g., Naïve Bayes or SVM). Analyze its performance using metrics such as accuracy, precision, recall, and F1-score.*

9.1 System Design

A Multinomial Naïve Bayes classifier is well suited to spam detection because email text can be represented as a bagof-words feature vector, and Naïve Bayes performs efficiently and effectively on high-dimensional, sparse text data.

Pipeline:

1. Data collection: a labeled corpus of emails (e.g., the SpamAssassin or Enron-Spam dataset), split into spam and ham (legitimate) classes.
2. Preprocessing: lowercase conversion, removal of stop words and HTML tags, tokenization, and stemming/lemmatization.
3. Feature extraction: TF-IDF or simple term-frequency vectorization of the cleaned tokens, optionally restricted to the top-N most informative terms.
4. Model training: fit a Multinomial Naïve Bayes classifier on a training split (e.g., 80%) using Laplace (additive) smoothing to handle unseen words.
5. Evaluation: test on a held-out 20% split, using stratified sampling to preserve the spam/ham ratio.

9.2 Evaluation Metrics

Given a confusion matrix with True Positives (TP, spam correctly flagged), False Positives (FP, ham incorrectly flagged as spam), False Negatives (FN, spam missed), and True Negatives (TN, ham correctly passed):

- Accuracy = (TP + TN) / (TP + TN + FP + FN) — overall correctness.
- Precision = TP / (TP + FP) — of messages flagged spam, how many truly are spam (critical, since a false positive means losing a legitimate email).
- Recall = TP / (TP + FN) — of all actual spam, how much was caught.
- F1-score = 2 × (Precision × Recall) / (Precision + Recall) — harmonic mean balancing both.

9.3 Representative Results

Illustrative results typical of Naïve Bayes spam filters evaluated on a balanced dataset:

Metric	Result (illustrative)
Accuracy	~97.2%

Precision	~96.5%
Recall	~94.8%
F1-score	~95.6%

9.4 Analysis

Naïve Bayes achieves strong performance with low computational cost, making it attractive for real-time filtering at mail-server scale. Precision is prioritized over raw recall in most deployments, since a false positive (legitimate mail marked spam) is typically more costly to the user than an occasional missed spam message. Comparing against an SVM classifier on the same data typically shows SVM achieving marginally higher accuracy and F1-score (often 1–2 percentage points) at the cost of significantly higher training time, especially with non-linear kernels — an important trade-off for systems needing frequent retraining on evolving spam patterns.

Question 10: *Design and evaluate an integrated email security solution that combines email header analysis, spam filtering, phishing detection, and digital signature verification. Analyze its effectiveness against modern email-based cyber threats.*

10.1 Integrated Architecture

The proposed system processes each incoming email through a multi-stage pipeline before delivery to the user's inbox, combining the mechanisms analyzed in the earlier questions into one defense-in-depth solution.

Stage	Component	Function
1	Header Analysis Engine	Parses Received chain, checks SPF/DKIM/DMARC results, flags From/Return-Path mismatches
2	Spam Classifier (Naïve Bayes/SVM)	Scores message content for spam likelihood using trained ML model
Stage	**Component**	**Function**
3	Phishing Detection Module	Checks embedded URLs against threat-intelligence blocklists, detects lookalike/typosquatted domains, inspects for urgency/credentialharvesting language patterns
4	Digital Signature Verification	Validates PGP/S/MIME signatures where present, confirming authenticity/integrity of signed messages
5	Policy Engine / Aggregator	Combines all stage scores into a composite risk score and applies action: deliver, quarantine, or reject

10.2 Composite Risk Scoring

Each stage contributes a weighted score (e.g., header anomalies 30%, spam classifier probability 30%, phishing indicators 30%, signature status 10% as a confidence booster). Messages exceeding a high threshold are rejected outright; those in a middle band are quarantined for user review; low-risk messages are delivered normally. This

layered scoring avoids relying on any single technique, since attackers who evade one layer (e.g., a spam filter) are still likely to be caught by another (e.g., header/DMARC failure or a known-malicious URL).

10.3 Evaluation Against Modern Threats

Threat	Layer(s) That Catch It	Effectiveness
Bulk spam	Spam classifier	High — ML content filtering is mature and highly accurate for generic spam
Domain spoofing	Header analysis (SPF/DKIM/DMARC)	High — cryptographic/policy checks reliably detect unauthorized senders
Spear-phishing with legitimatelooking domain	Phishing detection (URL/typosquat analysis) + header analysis	Moderate-to-high — depends on threat-intel freshness and domain similarity heuristics
Business Email Compromise (BEC) — no malware/links, pure social engineering	Phishing detection (language pattern analysis) only, weakly	Lower — hardest category, since headers may pass validly (compromised legitimate account) and content contains no obvious malicious artifact
Tampered/forged signed message	Digital signature verification	High — cryptographic signature failure is unambiguous

10.4 Analysis

The integrated approach substantially outperforms any single technique in isolation because modern email threats rarely rely on just one weakness: a well-crafted phishing email may pass a spam filter's statistical model while still failing DMARC alignment, or may pass header checks (when sent from a compromised legitimate account) while still containing a malicious link caught by threat-intelligence lookups. The residual gap is Business Email Compromise, where the sending account and infrastructure are entirely legitimate and the attack is purely social; addressing this category requires complementary controls beyond technical filtering, such as user awareness training and out-of-band verification for high-value transactions (e.g., wire-transfer requests).

10.5 Conclusion

Layered, integrated email security — combining cryptographic verification, statistical content filtering, threatintelligence-driven phishing detection, and header forensics — provides materially stronger protection than any individual mechanism, while acknowledging that no automated system fully eliminates the human-factor risk inherent in socially engineered attacks.